
Privacy-Preserving Bug Severity Prediction in Federated Software Repositories Using TF-IDF Features and Residual MLP

Presented By:

S. M Tasnimul Hasan - 1000060032

Md. Ashfaqur Rahman - 1000060031

Noholi Islam Duti - 1000058446

Miftaun Noor - 21241021

Introduction

What is Bug Severity Prediction?

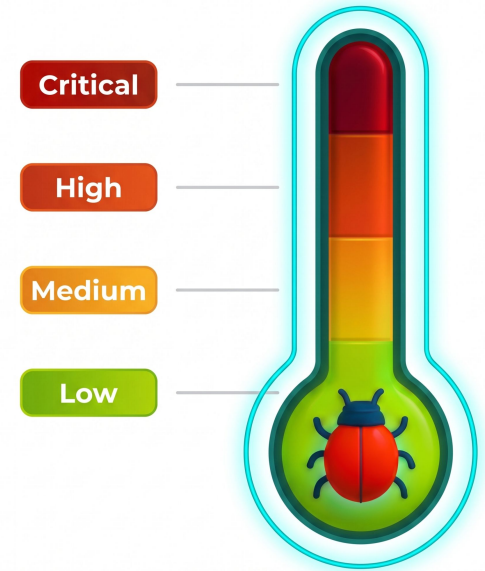
- Bug severity prediction is the task of classifying bugs based on their impact on the software system
- It assigns a severity level such as:
 - Critical
 - High
 - Medium
 - Low

Example:

- Bug A: Application crashes during user login → **Critical**
- Bug B: Payment gateway delay or transaction timeout → **High**
- Bug C: UI misalignment in settings page → **Medium**
- Bug D: Typo in help documentation → **Low**

Key Insight:

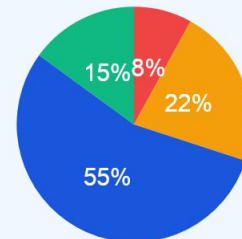
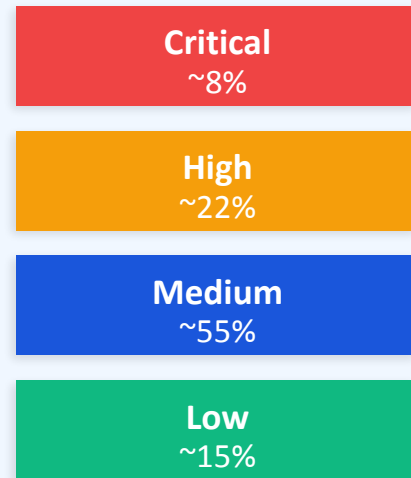
Bug severity prediction helps developers and QA teams prioritize critical issues first.



Dataset Overview

Source	HuggingFace — AliArshad/Bugzilla_Eclipse_Bug_Reports_Dataset
Key Fields	Short Description · Severity Label · Resolution Status · Project
Text length	Mean: 8 words · Median: 8 words · 95th percentile: 15 words
Split	Created 4 clients using IID partitioning (80% train / 20% test each)

Severity Distribution



Data Preprocessing

Step 01: Text Cleaning

Applied several preprocessing operations to normalize the textual data:

- Converted all text to lowercase
- Removed URLs and hyperlinks
- Replaced numbers with a common token (NUM)
- Removed special characters and punctuation
- Eliminated extra white spaces

Step 02: Severity Class Merging

Original Bugzilla severity labels were merged to reduce class imbalance and simplify classification.

Original Bugzilla Label	Mapped Class
blocker, critical	Critical 
major	High 
normal	Medium 
minor, trivial	Low 

Data Preprocessing

Step 03: Rich Text Construction

- Multiple informative fields were combined into a single textual feature.



Step 04: Data Filtering & Validation

- Removed rows with missing values
- Dropped empty or very short texts
- Kept only selected target severity classes

Step 05: Dataset Shuffling

- Randomly shuffled dataset
- Ensured unbiased training
- Improved reproducibility

Feature Engineering: TF-IDF Vectorization

What is TF-IDF?

- Measures the importance of a word in a document compared to the entire dataset.
- Higher weight to important words and Lower weight to very common words.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

TF = term frequency in a document | IDF = $\log(N / df(t))$

Example:

Bug: “application crash occurs during startup”

TF-IDF Weights: crash → 0.82, startup → 0.67, application → 0.21, during → 0.05

Key Insight:

The word “crash” receive higher importance because they strongly indicate severe bugs.

Feature Engineering: One-Hot Encoding (OHE)

What is One-Hot Encoding?

- One-Hot Encoding transforms categorical values into binary vectors, where:
 - 1 indicates the presence of the category
 - 0 indicates absence
- Each unique category becomes its own column in the feature matrix.

Example (Resolution Status):

Categories: [FIXED, NOT_FIXED]

- FIXED is encoded as [1, 0]
- NOT_FIXED is encoded as [0, 1]

Example (Project Feature):

Categories: [Bugzilla, CDT, Core, Firefox, JDT, Platform, Thunderbird]

- Firefox → [0, 0, 0, 1, 0, 0, 0]

Key Insight:

Categorical features can influence bug severity patterns.

Feature Engineering

Final Feature Vector



Final Representation:

TF-IDF (5 000) $\oplus$ OHE (8) = 5 008 dimensions

Output:

Final vector is used as input to **Residual MLP**

IID Partitioning Across Clients

What is IID Partitioning?

In IID partitioning, each client receives data with a similar statistical distribution and class proportion.

Key Characteristics:

- Random and homogeneous data distribution
- Similar class balance across all clients
- Simulates ideal federated learning conditions

Partitioning Procedure

- Random Dataset Shuffling
- Stratified Client Splitting
 - Client 1 → Critical: 25%, High: 25%, Medium: 25%, Low: 25%
 - Client 2 → Critical: 24%, High: 26%, Medium: 25%, Low: 25%
- Local Train-Test Split
 - 80% Training Data
 - 20% Testing Data

IID Data Distribution - 4 Clients

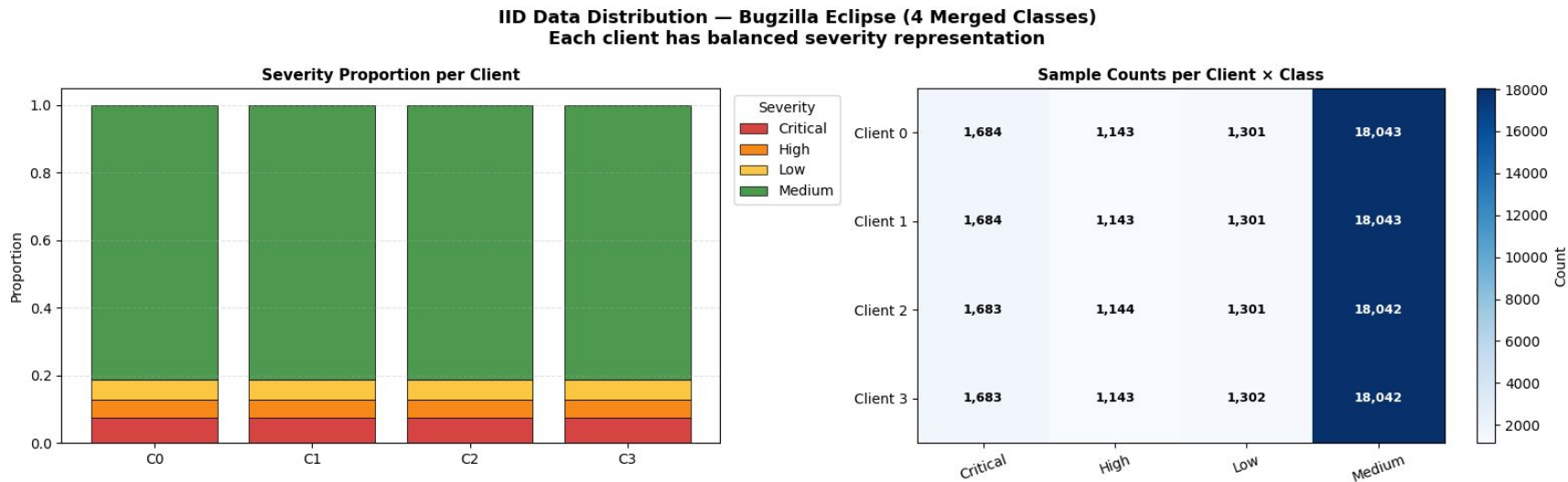


Figure: IID Data Distribution

BugSeverityMLP (Residual MLP)

A Multilayer Perceptron (MLP) is a foundational deep learning feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. It uses fully connected nodes, non-linear activation functions (e.g., ReLU, GeLU), and backpropagation to model complex, non-linear relationships.

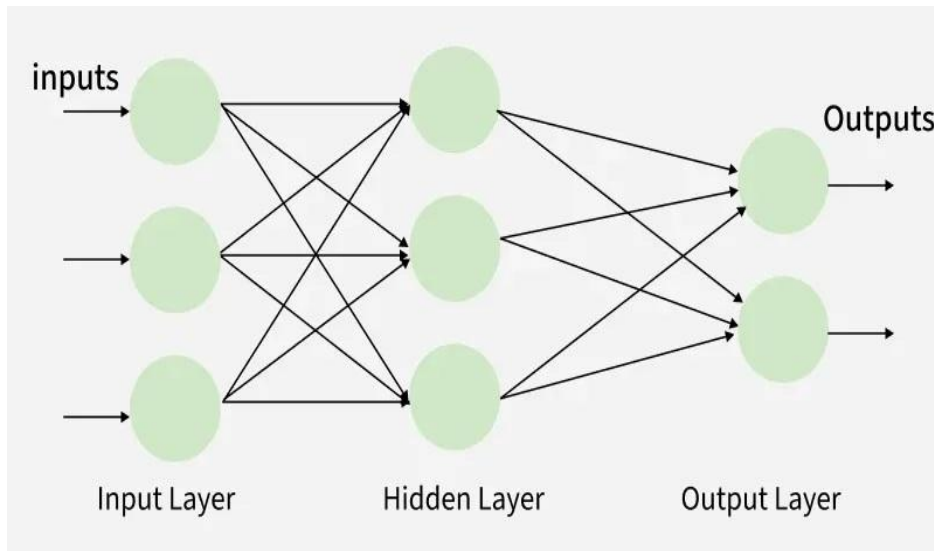


Figure: MLP

BugSeverityMLP (Residual MLP)

1. Forward Propagation

In the proposed Bug Severity Prediction system, the input bug reports are first converted into numerical feature vectors using TF-IDF and categorical feature encoding. These feature vectors are then passed through the Multi-Layer Perceptron (MLP) network.

The forward propagation process follows these steps:

Mathematically, each neuron computes:

$$Z = Wx + b$$

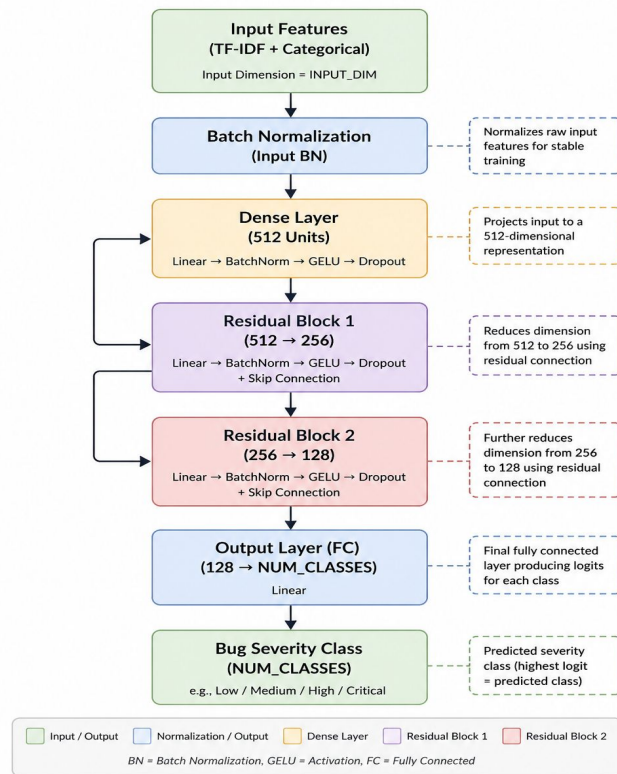
An activation function (GELU) is then applied:

$$a = \text{GELU}(z)$$

The final output layer produces logits for each severity class:

$$y = f(x)$$

BugSeverityMLP Architecture
(TF-IDF + Categorical Features)



BugSeverityMLP (Residual MLP)

1. Forward Propagation

Example:

Input Bug:

"Application crashes during login"

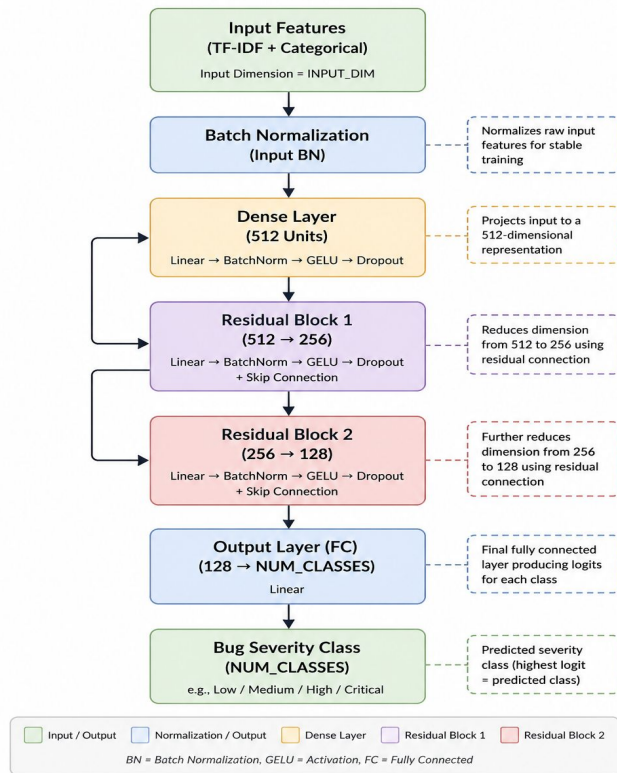
Predicted Output:

[Low: 0.02, Medium: 0.08, High: 0.81, Critical: 0.09]

Final Prediction:

High Severity

BugSeverityMLP Architecture
(TF-IDF + Categorical Features)



BugSeverityMLP (Residual MLP)

2. Loss Function

To measure prediction error, the model uses the Cross-Entropy Loss function, which is widely used for multi-class classification tasks.

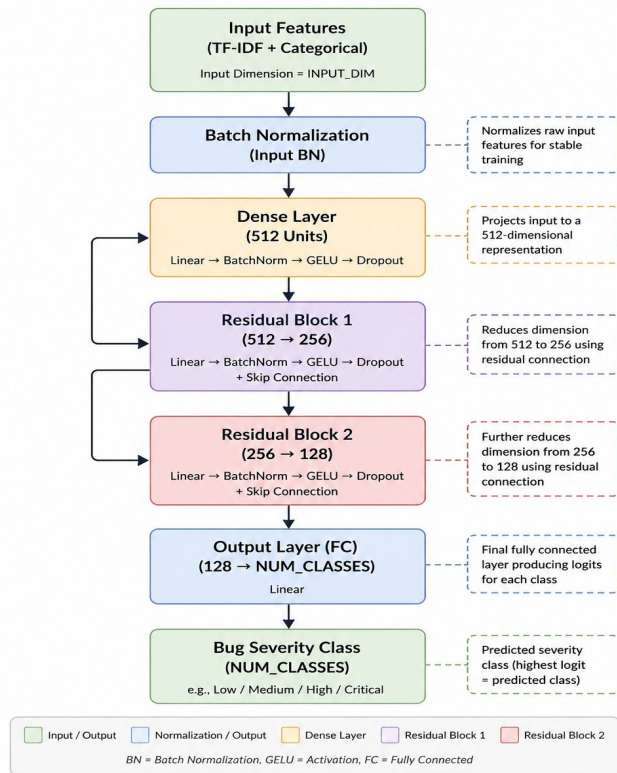
The loss function compares:

- actual bug severity label
- predicted probability distribution

The Cross-Entropy Loss is defined as:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

BugSeverityMLP Architecture
(TF-IDF + Categorical Features)



BugSeverityMLP (Residual MLP)

3. Backpropagation

Backpropagation is the learning mechanism of the neural network. After computing the loss, the model calculates gradients of the loss with respect to each weight parameter.

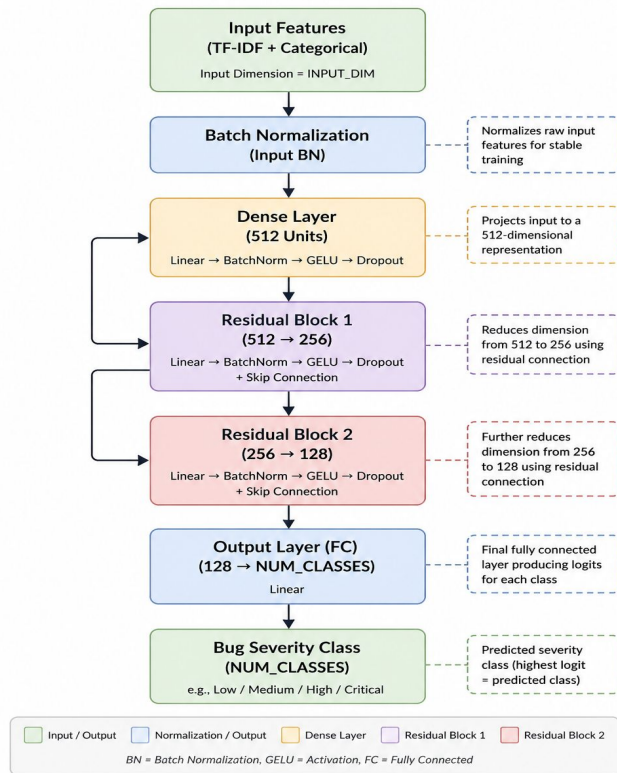
The gradients are computed using the chain rule of calculus:

$$\frac{\partial L}{\partial W}$$

In the proposed MLP architecture:

- gradients flow through residual connections
- residual blocks help prevent vanishing gradient problems
- training becomes more stable and efficient

BugSeverityMLP Architecture
(TF-IDF + Categorical Features)



BugSeverityMLP (Residual MLP)

4. Optimization

After gradients are computed, the optimizer updates the network weights to minimize the loss function.

The project uses an optimization algorithm such as Adam Optimizer.

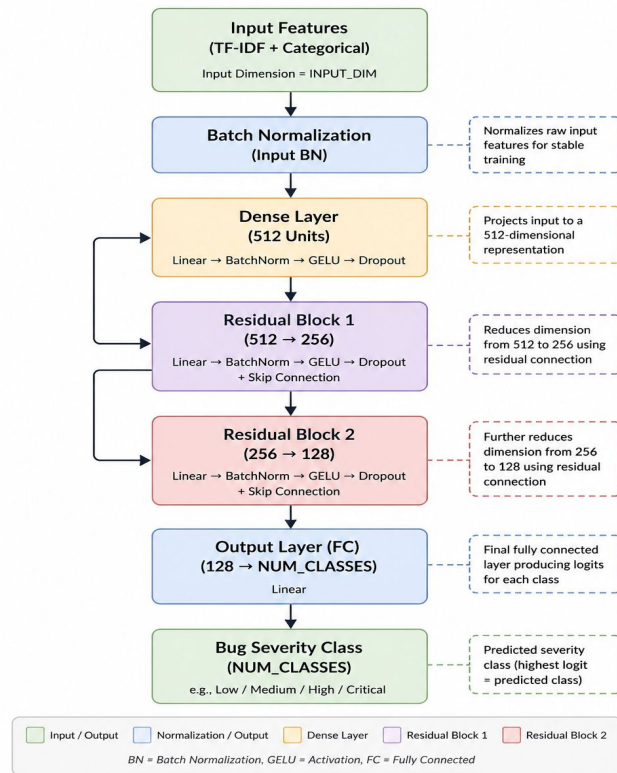
Weight update rule:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

The Adam optimizer is preferred because:

- faster convergence
- adaptive learning rates
- stable training performance

BugSeverityMLP Architecture
(TF-IDF + Categorical Features)



Proposed Model — BugSeverityMLP (Residual MLP)

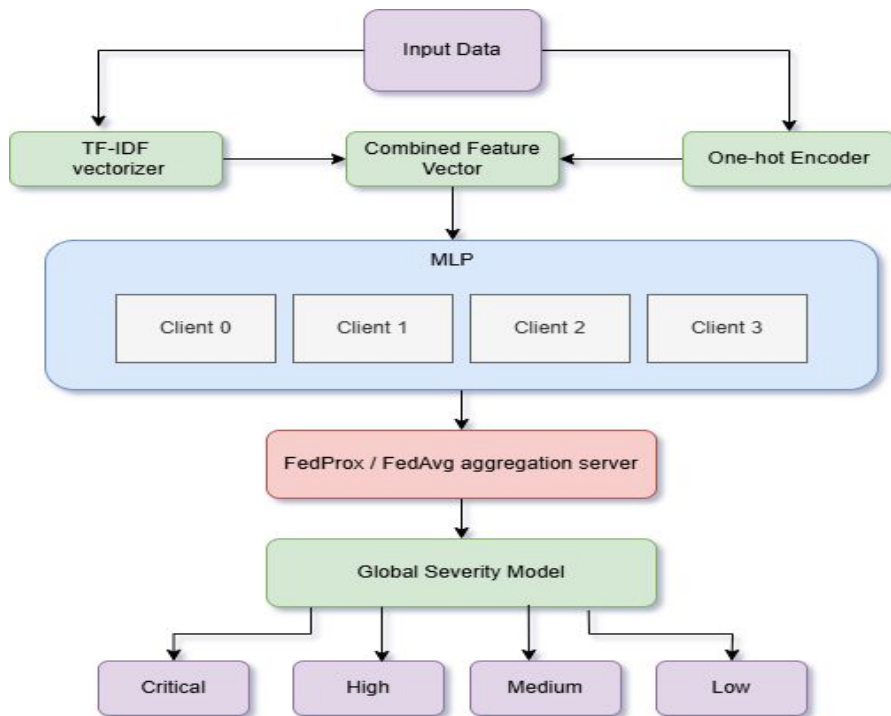
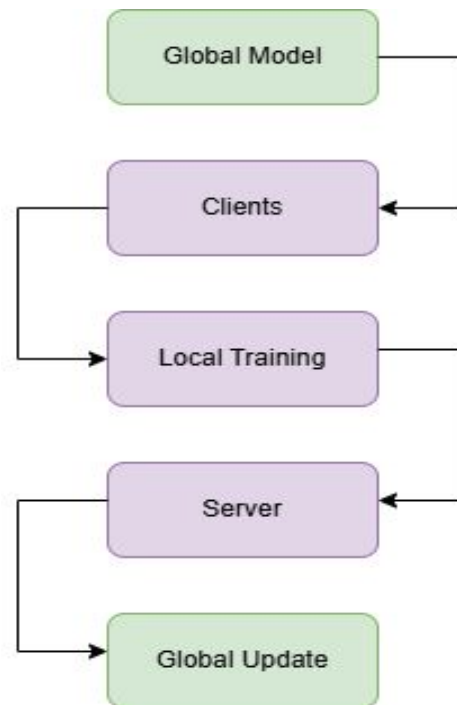


Figure: Proposed Model

Federated Learning

Key Idea of Our FL System

- No single centralized model training
- Same MLP model is distributed to all clients
- Each client trains independently on local bug data
- Only model updates are shared with the server



What is Federated Learning?

- Federated Learning is a distributed machine learning approach
- Models are trained across multiple devices or servers
- Each client uses its own local data for training
- Raw data never leaves the local device
- Only model updates are shared with a central server
- Enables collaborative learning without data centralization
- Organizations can collaboratively train models without sharing internal bug data

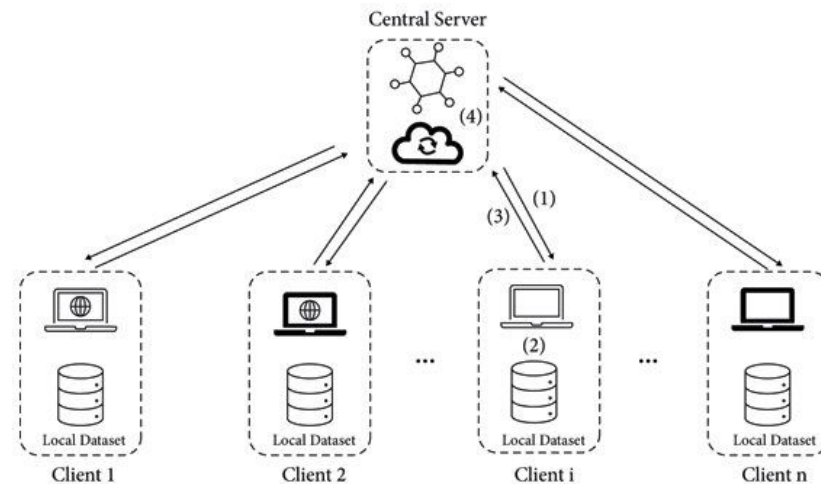


Figure: Federated Learning Architecture

Local Training at Each Client

Local Model Setup

- Each client contains a local copy of the same MLP model
- Model architecture is identical across all client

Training Process

- Global model is distributed to all clients in each communication round
- Each client trains independently using only local bug report data
- 4 clients simulate distributed software repositories
- Each client performs 8 local training epochs per round

Implementation

- Implemented using **Flower Federated Learning Framework**

FedProx Optimization

- FedProx is used during local client-side training
- Prevents local models from drifting too far from the global model
- Improves training stability and convergence
- Makes federated optimization more robust

Objective Function:
$$\min_w h_k(w) = F_k(w) + \frac{\mu}{2} \|w - w^t\|^2$$

In our Framework -

- Each client trains the local MLP using FedProx
- The proximal term keeps local updates closer to the global model

Server-Side Aggregation (FedAvg)

Federated Averaging (FedAvg) is a method used to aggregate model updates from multiple clients into a single global model

- After local training ,
 - Each client sends updated model parameters to the server
 - No raw data is shared between clients
 - The server aggregates all client updates using **Federated Averaging (FedAvg)**

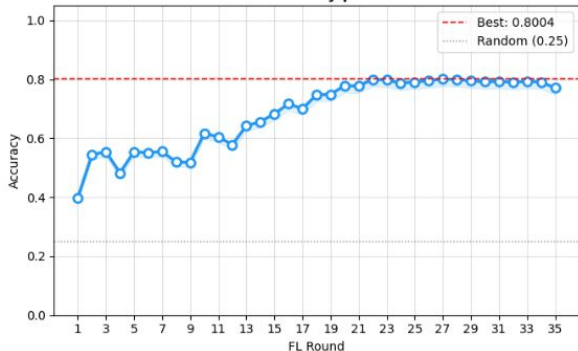
$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_k$$

- It computes a weighted average of all client model parameters based on their local data size
- The updated global model is sent back to all clients
- This process repeats for multiple communication rounds to improve performance

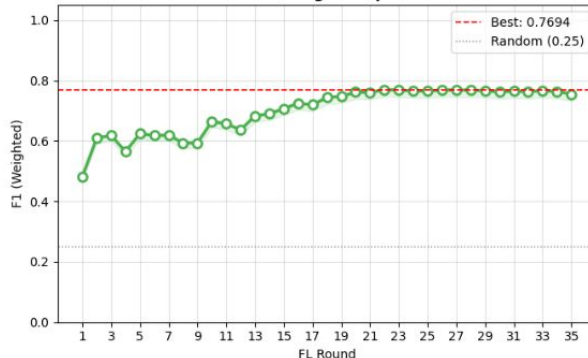
FL Training Curves — FedProx Result Analysis

TF-IDF MLP | 4 IID Clients | 35 Rounds | 8 Local Epochs | Bugzilla Eclipse

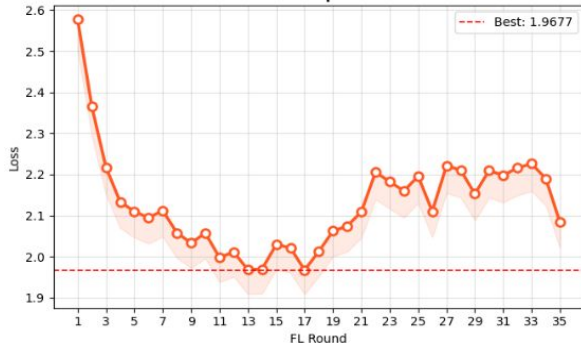
Global Accuracy per Round



Global F1 (Weighted) per Round



Global Loss per Round



Best Accuracy: 0.8004

Global accuracy reaches 80.04%, showing strong overall classification performance.

Best Weighted F1: 0.7694

Weighted F1 remains strong, indicating balanced performance across the merged severity classes.

Best Round: 27 / 35

The model achieves its best performance around round 27 during the federated training process.

Lowest Loss: 1.9677

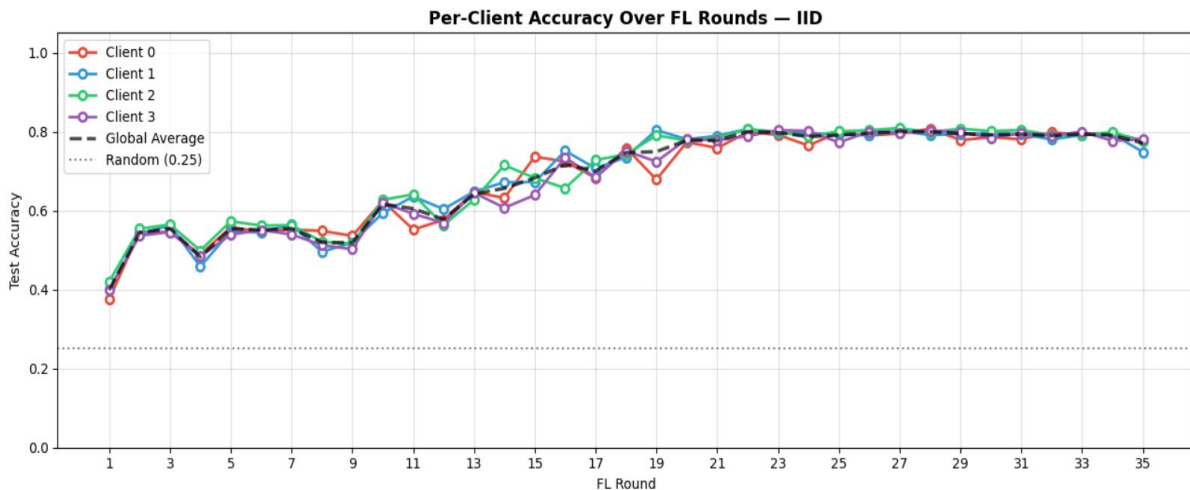
Training loss drops quickly in the early rounds and later becomes more stable.

Stable Convergence

Accuracy and F1 become steadier in later rounds, showing effective convergence with FedProx.

Takeaway: FedProx with TF-IDF MLP improves steadily and reaches stable convergence, with the best results around round 27

Per-Client Accuracy Over FL Rounds — IID Analysis



All Clients Improve Steadily

Each client rises from about 0.4 in the early rounds to nearly 0.8 by the end of training.

Strong Client Consistency

The four client curves stay close to each other, showing balanced learning under IID data partitioning.

Stable Convergence After Round 20

Accuracy becomes more stable in the later rounds, indicating convergence of the federated model.

Global Average Tracks Closely

The dashed global average closely follows the client trends, suggesting effective aggregation across all clients.

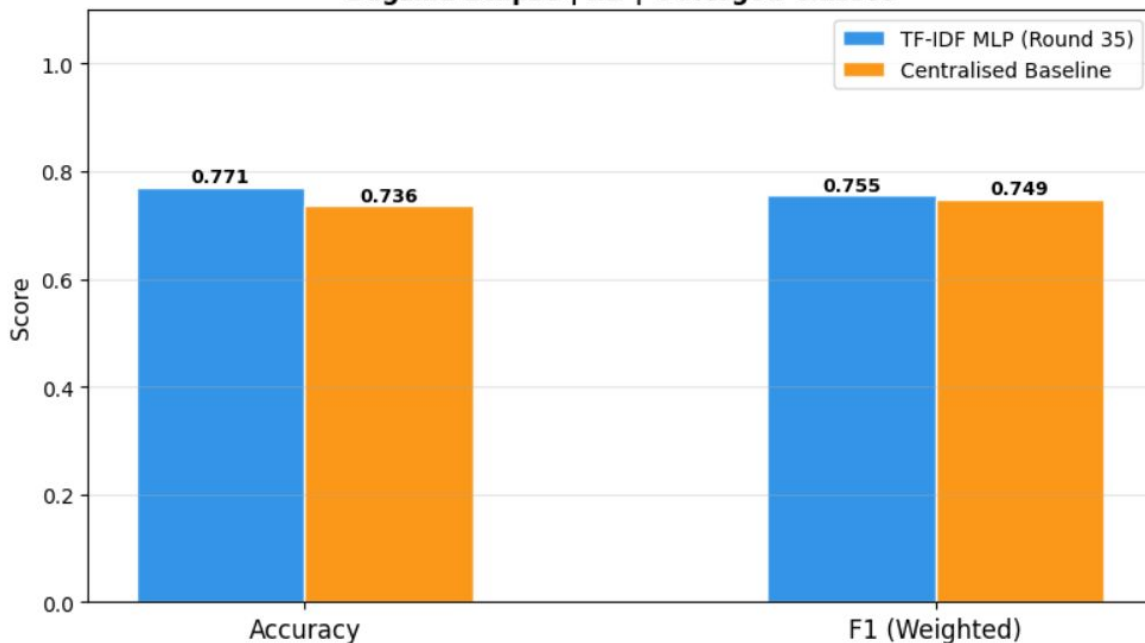
Far Above Random Baseline

All client accuracies remain well above the random baseline of 0.25 after the initial training rounds.

Takeaway: Under IID data, FedProx delivers stable and uniform client performance, with all clients converging near 0.8 test accuracy.

Federated Learning VS Centralised Bar Chart

TF-IDF MLP vs Centralised Baseline
Bugzilla Eclipse | IID | 4 Merged Classes



FL Achieves Higher Accuracy

TF-IDF MLP with federated learning reaches 0.771 accuracy, compared to 0.736 for the centralised baseline.

Weighted F1 Also Improves

The FL model achieves 0.755 weighted F1, slightly higher than the centralised baseline score of 0.749.

Accuracy Gain is More Noticeable

The accuracy improvement is about +0.035, showing that the FL model predicts the final classes better overall.

F1 Gain is Smaller but Positive

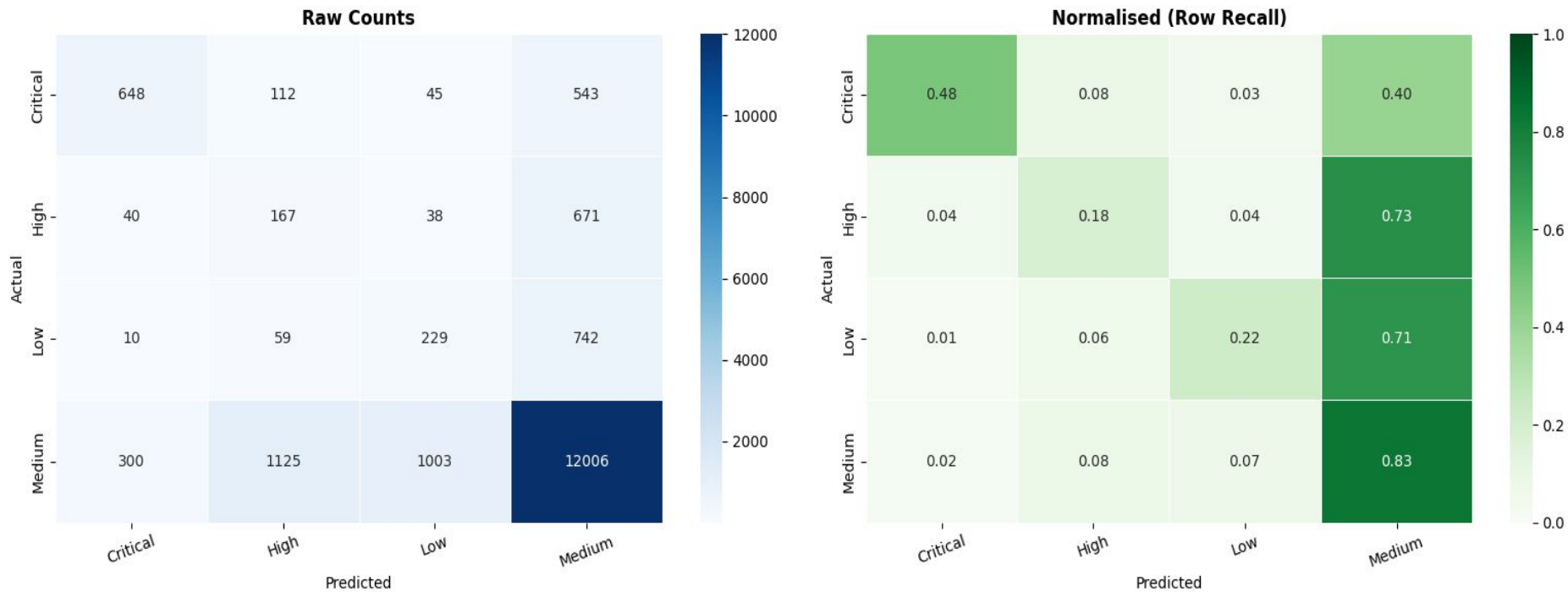
The weighted F1 improvement is about +0.006, meaning the class-balanced performance also improves slightly.

FedProx Performs Competitively

Even without centralising all data, the federated model performs better than the centralised baseline in this experiment.

Confusion Matrix Analysis

Confusion Matrix — TF-IDF MLP | Bugzilla Eclipse
4 Merged Severity Classes



Future Work

- Evaluate the framework under non-IID client data distributions.
- Apply Differential Privacy techniques for stronger privacy guarantees.
- Use transformer-based models such as CodeBERT or BugBERT for feature extraction.
- Extend the system to multi-task prediction (severity, priority, resolution time).
- Compare different federated aggregation methods such as FedAvg, FedAdam, and FedNova.
- Expand the dataset using larger cross-project and multilingual bug repositories.
- Test the framework in real-world software development environments.

Conclusion

- Federated learning successfully predicted bug severity while preserving client data privacy.
- TF-IDF + bigrams outperformed neural embeddings for short bug titles.
- FedBugSev achieved performance close to centralized learning with lower privacy risk.
- Residual MLP proved efficient and FL-friendly due to fast convergence and low overhead.
- FedProx improved training stability during federated optimization.
- Metadata features (e.g., resolution status) boosted prediction accuracy.
- Overall, the framework demonstrates that privacy-preserving bug severity prediction is both practical and effective.

Thank You

Any Questions?

